TECHNICAL REPORT · HACK APERTUS

Mandate Desk

Mandate-bounded payment agents for Swiss fiduciaries

Track **Track 2B — Mandate Desk** Event **Online** Team **hack-1 — Sohee Lee** Demo `link to video`

SUMMARY

Swiss fiduciaries pay supplier invoices for many small clients, each of which delegated a different scope and limit. Mandate Desk gives each client its own payment agent and keeps it inside the client's signed mandate: Apertus 8B reads and reviews multilingual invoices, code enforces the mandate and checks what Apertus read, a person decides the exceptions, and every step is signed into a ledger. On 500 unseen synthetic invoices run once, Apertus alone decided to pay **178 of 300** invoices it should not have; with the harness, **0 of 300 (95% CI 0–1.3%)**, holding 3.5% of legitimate invoices for a person. Seventeen attacks on the agent's identity and authority were blocked in 561 of 561 attempts (scenarios we wrote; not a security audit).

CONTENTS

1. Introduction
2. System design
   - 2.1 Architecture
   - 2.2 Target architecture (mandatory)
   - 2.3 Use of Apertus
   - 2.4 Agent identity, gate and record
   - 2.5 Agent Health: what the firm sees
3. Data
4. Evaluation
   - 4.1 Task, metric and baseline
   - 4.2 Reference test set
   - 4.3 Blind study: three fresh sets, each run once
   - 4.4 Agent verification
   - 4.5 Risk score (data-science layer)
5. Discussion
6. Limitations
7. Conclusion
8. Reproducibility

License

References

1 Introduction

Problem. Around 12,900 Swiss firms do accounting, bookkeeping and tax work (NOGA 6920; secondary source, Valindex 2026-02), most with 1–9 employees. A daily task is paying supplier invoices for many small clients, each of which delegated a different scope and limit. A language model can read the invoices, but on its own it *fails open*: on our test set Apertus alone paid 33 of 45 invoices it should not have, asserting facts it did not check ("IBAN matches on file" for a broken checksum) and never summing the month.

Question. Can a harness around Apertus 8B keep each client's payments within that client's signed mandate, so that it pays no invoice it should not, while a person decides the exceptions and every action can be checked afterwards?

Approach. Mandate Desk works on four axes:

- **Mandate and authority.** Each client signs a mandate (scope, limits, validity, delegate = the firm). Each client gets its own agent with its own key and DID, a firm-signed ID card and a badge derived from the mandate. Every request the agent makes is signed and checked by a gate (five checks) before it takes effect.
- **Harness.** Apertus reads multilingual invoices and reviews their content; code enforces the mandate and checks that what Apertus read is written in the invoice, where Apertus quotes it.
- **Human control.** Exceptions and uncertain payments go to a person, who approves (a one-time grant bound to client, invoice, IBAN and amount), rejects, or stops the agent with a kill switch. Overrules become lessons that Apertus writes into the firm manual.
- **Signed record.** Every request, its outcome and every human intervention is in a hash-chained, signed ledger that keeps the request the agent signed, so an auditor can re-check it. Payments leave only as an ISO 20022 pain.001.001.09 file per client.

Outline. Section 2 describes the system, Section 3 the data, Section 4 the evaluation against Apertus alone, Section 5 what the results suggest, Section 6 the limitations and Section 7 concludes.

2 System design

2.1 Architecture

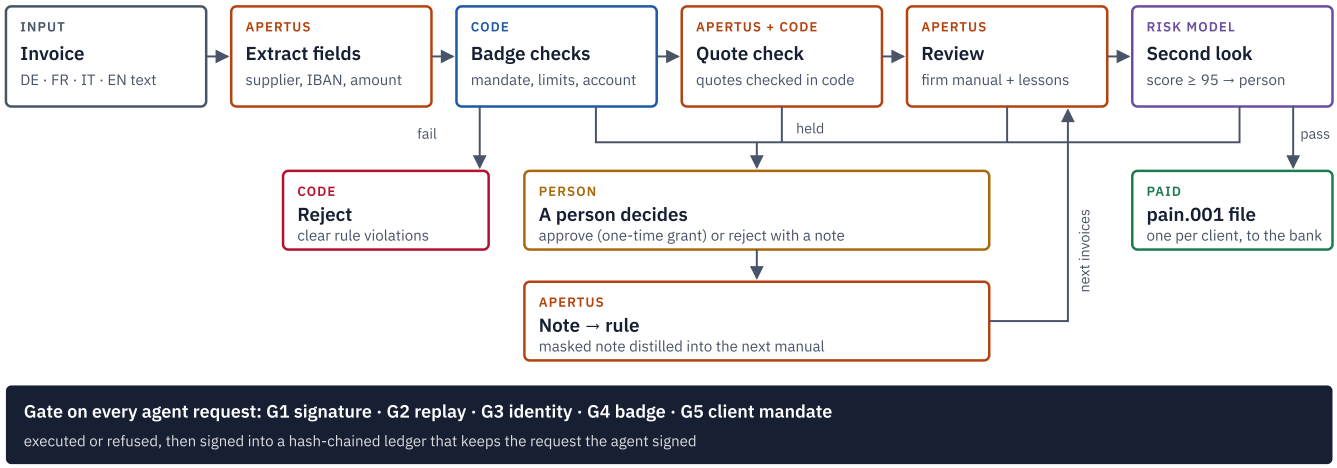


Figure 1. One invoice through the harness. **Apertus** reads, quotes and reviews; **code** enforces the mandate; **the risk score** takes a second look; **a person** decides the exceptions. Every agent request passes the gate and is signed into the ledger.

All code is under `src/agent/` and runs in pure Python except the Apertus calls: synthetic data (`datagen.py`), the Apertus client with a record/replay cache (`llm.py`, `prompts.py`), badge, verification and the client-signed mandate (`badge.py`, `credential.py`), decision paths and lessons (`decide.py`, `lessons.py`), identity, gate and desk (`identity.py`, `gate.py`, `desk.py`), risk score and attack scenarios (`risk.py`, `security.py`), and the signed ledger, payment files and reviewer web app (`ledger.py`, `pain001.py`, `app.py`, FastAPI on port 8000).

2.2 Target architecture (mandatory)

a) On-premise, with **b) air-gapped** operation possible: a fiduciary holds client bank details, salaries and supplier relationships, so the design keeps everything on the firm's own infrastructure.

- **Runtime:** one container (Python 3.11, FastAPI, Uvicorn, cryptography) and one OpenAI-compatible endpoint serving Apertus 8B, run by the firm or a Swiss hosting partner (only the Public AI API was tested). No other network access: no payment provider, no external API. Payments leave only as a pain.001 file the firm uploads to the client's bank.
- **Build time:** the `python:3.11-slim` image, three pip packages, Apertus weights from Hugging Face; all can be mirrored internally, so the runtime can be air-gapped.
- **During development** we used the Public AI inference API because the organiser's CSCS key was not at hand. The endpoint is read from `LLM_BASE_URL`; CSCS and a self-hosted server were not tested. Recorded responses make the evaluation independent of the endpoint.

2.3 Use of Apertus

- **Model:** Apertus v1.5 8B (`swiss-ai/apertus-v1.5-8b` on the endpoint; weights `swiss-ai/Apertus-v1.5-8B`), inference only, structured JSON outputs, temperature 0, no fine-tuning, no other model (not even as a judge).
- **Five roles:** (1) extract fields from DE/FR/IT/EN invoices; (2) the *baseline*: the whole decision in one prompt; (3) review content after the hard checks; (4) for payment candidates, quote where it read IBAN, invoice number and total (checked in code); (5) distil masked reviewer notes into manual rules.
- **Where it runs:** Public AI inference API (OpenAI-compatible). Reference run and holdout: 501 recorded responses, latency median 3.3 s / p90 7.3 s, well under USD 0.10; study: 7,658 responses, about USD 0.50. Every response is recorded with its exact request (`data/cache/`), so judges replay the same run without a key.

2.4 Agent identity, gate and record

The harness answers *what* the agent may do; this layer answers *who* did it and lets anyone check. The concept follows KYA-OS (Know Your Agent OS: identity, delegation, signed proof, ledger), built by Vouched and donated to the Decentralized Identity Foundation in March 2026, whose reference implementation `@kya-os/mcp` is MIT-licensed; the author's earlier "Agentic ID" project built on it and added the lesson layer. This project re-implements the ideas in pure Python and contains no KYA-OS code.

Principal	Key	Signs
Client company	own Ed25519 key (did:key, recorded at onboarding)	the mandate: scope, limits, validity, delegate = the firm
Fiduciary firm	firm key	each agent's ID card (duty, the client manager who manages it, head of payments, *is AI*) and badge (actions, its one client, cap, validity)
Agent: one seat per client, one key per generation	own key → DID	every request: action, client, payload hash, nonce, time
Ledger	ledger key	every entry, on top of the hash chain

The **gate** checks each request: G1 signature and payload hash; G2 nonce unseen and request fresh; G3 firm-issued ID card, agent not revoked; G4 the badge covers action, client, date and amount (a payment needs a positive amount, a case and an IBAN; above the cap only with a human grant for exactly this payment, used up once; a "pay" that is not marked as executing a payment is refused); G5 the client's mandate is validly signed, names the firm, is in force, and the amount is within its single limit unless a human granted it. One seat per client means one client's agent cannot act for another and data never mixes. **One-click deployment** checks the mandate, issues the card, derives the badge from the mandate, fixes the manual and self-tests against the gate before the agent goes on duty; the **kill switch** revokes one seat only; **replacing** an agent issues a new key while the seat's lessons carry over.

A payment decision and its payment file are one step: the file is written first; if it fails validation, cannot be written or the gate refuses it, the decision is voided and the invoice waits for a person. Each payment is its own batch with its own message ID; an issued file is never rewritten. In the web app, this month's payments come from the ledger and the hard checks and the same-supplier-and-amount check run again under a lock just before paying.

The **ledger** opens with the firm and ledger DIDs, every client mandate with its signature, and each agent generation's card, badge and manual fingerprint. Each request is kept whole (signed stamp and payload) and the recorded fields are taken from the payload. An auditor can check from the ledger alone that every executed action was signed by an agent the firm issued and had not revoked, and that the entry says what was signed (`audit_identities`; reference run 78 of 78). FINMA Guidance 08/2024 calls for the identification, assessment, management and monitoring of AI risks; the EU AI Act requires automatic logs (Art. 12) and human oversight including a way to stop the system (Art. 14) for high-risk AI systems. We did not assess whether this use is high-risk; this is not a compliance assessment.

2.5 Agent Health: what the firm sees

Agent Health, the reviewer app's firm-level screen, has three tabs; each answers one question, and they build on each other (the other firm screens: Audit Trail verifies the signed ledger, Invoice Pool lists the data the checks use):

Tab	What it evaluates	Why it is needed
Risk score (data science)	how the risk score is built and how good it is: 3,000 synthetic past payments, 13 features, split, L2 choice, ROC and PR curves, confusion matrices, errors	the score is one part of the harness; its quality has to be shown before its second look is trusted
Who caught what	on the 78 labelled test invoices, which layer caught each risk type (code checks, Apertus review, risk score, missed); from the ledger, signals across clients (one IBAN on several suppliers or clients, unusual amounts paid automatically)	shows which part does the work and where the gaps are, and what a client manager should look at
Results	the whole system against Apertus alone: wrong payments step by step on the test set, the confirm set and the holdout, attacks blocked, and each client's agent	shows whether the harness as a whole pays fewer invoices it should not

3 Data

All data is **synthetic and fictional**, generated by templates (not a model), so labels are exact; no personal data; licence CDLA-Permissive-2.0. Reference set (seed 2026): 5 client mandates (one revoked), 19 suppliers, 120 invoices (DE 37 / FR 33 / EN 27 / IT 23), two layouts per language, Swiss number formats, VAT 8.1% or exempt, one planted fault per non-payable invoice in 11 types (revoked, out of scope, invalid IBAN, duplicate, single and monthly limit, changed IBAN, new supplier, pressure, VAT mismatch, wrong addressee). Split: 42 train (only to collect reviewer notes), 78 test; a 42-invoice holdout (seed 2027). Study sets (seeds 3031–3033): four layouts (adding e-mail and a Swiss QR-bill payment part), two faults on one invoice, and unreadable fields (no invoice number, no IBAN, illegible total); per 100: 40 payable, 45 escalate, 15 reject.

4 Evaluation

4.1 Task, metric and baseline

Task: pay / escalate / reject per invoice. **Primary metric:** wrong payments, non-payable invoices the system decided to pay on its own. No money moves in any evaluation; in operation a "pay" puts the invoice into the pain.001 file for the bank. Also: wrongly paid CHF, missed payments (payable invoices held), accuracy. Model failures fail closed (escalate) and are counted.

Baseline: Apertus alone with the *same* information (mandate, supplier list, this month's payments, invoice) and the same written policy and manual gen 0. Harness rows from "manual gen 1" on also use lessons; the step-by-step rows separate the two, and a baseline given manual gen 1 too (Section 4.3) shows the lessons barely help on their own.

4.2 Reference test set

78 invoices: 33 payable, 45 not.

Setup	Accuracy	Wrong payments (of 45)	Wrongly paid CHF	Missed (of 33)
Baseline: Apertus alone	0.513	33	138,781	1
Apertus extraction + badge	0.872	10	34,521	0
+ review, manual gen 0	0.872	10	34,521	0
+ review, manual gen 1 (lessons)	**0.910**	**7**	**26,771**	**0**
+ two lessons promoted to hard checks ✱	0.974	2	9,290	0
+ verification (written, quoted, duplicate) ✱	0.987	1	4,650	0
+ risk-score second look (≥ 95 → person) ✱ = production	**1.000**	**0**	**0**	**0**

✱ Added after seeing these results, so not blind; the study below is. A fresh **holdout** of 42 invoices, run once with the design frozen up to the promoted lessons, gave 14 → 0 of 25 with no missed payment.

4.3 Blind study: three fresh sets, each run once

Each step was committed before the set it was tested on was run.

1. **Tune (500)** chose the second-look threshold by a rule fixed in advance (fewest wrong payments while holding at most 2% more legitimate invoices). It showed Apertus filling gaps (16 of 20 illegible totals computed from net + VAT and paid), so we added a check that the IBAN and total read are written in the invoice.
2. **Final (1,000)**, blind for that design: Apertus alone paid 337 of 600; the harness **29 (4.8%, CI 3.4–6.9%)**, every one an invoice *without an invoice number* that Apertus filled with the payment reference or the word "RECHNUNG". All other types were never paid.
3. Instead of a list of labels, Apertus now **quotes the exact words where it read IBAN, invoice number and total**, and code checks each quote is in the invoice and contains the value, and that the invoice number has a digit and is not the reference; same supplier and amount already paid this month is a possible duplicate. Re-scored, tune and final drop to 0 (not blind); the threshold rule picks 95.
4. **Confirm (500, seed 3033)**, blind for the final design. A first run with the earlier design was stopped at invoice 264 before any scoring when step 3 was decided; its recorded responses were reused.

System (confirm set)	What changed	Who decides	Wrong payments (of 300)	Wrongly paid CHF	Missed (of 200)
Apertus alone	all rules in one prompt	Apertus	178	642,157	2
Apertus alone, manual gen 1	same, with the lessons	Apertus	171	681,234	0
extraction + badge	Apertus reads; mandate, limits, account checked in code	code	89	226,007	5
+ review, manual gen 0 → gen 1	Apertus reviews what passed; lessons written by Apertus	code, then Apertus	88 → 67	172,827	5
+ promoted lessons	VAT mismatch and wrong addressee move into code	code	31	67,960	5
+ verification	Apertus quotes, code checks the quotes	Apertus and code	0	0	7
+ second look at 95 = production	high risk score → person	statistical model	**0 (CI 0–1.3%)**	**0**	**7**

The production harness paid 193, sent 232 to a person and rejected 75. **Who stopped what** (first layer that held each non-payable invoice; rejected / to a person): hard checks 185 (75 / 110), promoted lessons 39 (0 / 39), verification 56 (0 / 56), Apertus review 20 (0 / 20), second look 0. Only clear rule violations are rejected in code; everything else goes to a person. The 7 legitimate invoices held are reading errors caught before money moves (5 misread IBANs, 2 quotes that did not match); no paid invoice carried a wrong IBAN or amount. The final set gives the same split (63 / 12 / 18 / 6%). How accurately and quickly people handle the 232 was not measured.

4.4 Agent verification

Each of 17 attack scenarios was replayed for each of the 33 paid invoices against a freshly hired agent (`security.py`, no model calls); 17 × 33 = 561 attempts, all blocked (scenarios we wrote; not a security audit). **G1** caught a forged signature and an amount or IBAN changed after signing; **G2** a replayed and a stale (1 h) request; **G3** an unknown agent key and an agent revoked by the kill switch; **G4** an agent of another client, an amount above the cap without a grant, an expired badge, a reused grant, another client's grant, a negative, zero or NaN amount, and a "pay" of CHF 9,999,999 marked as

not executing a payment; **G5** tampered mandate limits, a mandate naming another delegate, a revoked mandate, and an amount above the client's limit with the badge cap set too high. The legitimate control request was accepted 33 of 33 times. In the reference run the gate accepted 78 requests from active clients (74 payment decisions and 4 payment files) and refused the 4 for the client that had revoked its mandate; a request signed for CHF 100 can no longer be recorded as CHF 999,999 (`tests/test_core.py`).

4.5 Risk score (data-science layer)

L2-regularised logistic regression (Newton/IRLS, pure Python) on 3,000 synthetic past payments with 13 features (amount z-score per supplier, changed or invalid IBAN, new supplier, round amount, sum mismatch, wrong addressee, urgency words in four languages, short due date, duplicate, limits, scope, month load); L2 weight chosen on a 20% validation split. Fraud ROC AUC 0.970 on the test set and 0.984 on the holdout (unsupervised baseline 0.878 / 0.838). It only sends a payment the harness would make to a person (score ≥ 95); on the reference set it caught the one pressure invoice the reviewer missed, on the confirm set it added nothing. Every step is shown on Agent Health's Risk score tab.

5 Discussion

Apertus alone fails open. It asserts facts it did not check and never sums the month, yet misses only 1 legitimate invoice on the test set, so the problem is verification, not caution. Giving it the lessons does not fix this: on the confirm set it still paid 171 of 300.

Rules belong in code, reading belongs to the model. The badge removes every error a rule can see, with no missed payment; extraction in four languages was near perfect (IBAN, invoice no., supplier 1.000, total 0.987). On the confirm set the hard checks held 185 of the 300 non-payable invoices and the Apertus review 20.

Lessons help where the signal is in the words, not with arithmetic: pressure invoices were caught 0 → 3 of 4, but the 8B reviewer asserts "correct totals" without adding, so those two lessons were promoted to code. Soft control finds the rule, hard control enforces it.

Each gap was found by a fresh set. The quote check exists because the final set showed 29 of 600 paid invoices without an invoice number; only the confirm set tests the final design blind.

6 Limitations

- Synthetic text invoices, one generator family; no OCR or scans. The 0 of 300 is evidence for the mechanism, not a rate to expect in production; each verification check was added after a set showed its gap, and only the confirm set tests the final design blind.
- The quote check proves a value is written in the invoice, not what it means (a reference line quoted as the invoice number passes if Apertus returns no payment reference; not seen on the confirm set). For a registered supplier the expense category comes from the register, not the invoice.
- The 8B model misread the IBAN on 64 of the 1,920 study invoices that carry one (3.3%); such invoices are held, which costs reviewer time.
- A changed bank account is held for a person; confirming it with the supplier on a known number is not built into the app (the queue only reserves two days for it when it computes how much time is left).
- When every check passes, Apertus's "pay" becomes a payment without a person; only the repeated hard and duplicate checks and the risk-score second look come after it.
- One risk model, fitted on all clients' history, and one threshold (95) serve every client; its features are measured against each client's mandate and usual amounts, but nothing is calibrated per client.
- Identity uses real Ed25519 and did:key but simple JSON documents, not a standard credential format; demo keys come from fixed seeds (production: HSM for firm and ledger keys); the ledger is not anchored externally.
- The web app has no login or roles; the ledger records who acted by role, not by a verified login. pain.001 files are checked structurally; an external reviewer reported that the four reference files pass the SIX pain.001.001.09 schema, which we could not reproduce; bank acceptance was not tested.
- Known display gaps in the app (the decisions and the ledger are right): "Paid / monthly limit" sums all standing payments, not only this month's; "Paid automatically" in CHF includes human-approved payments; one decisions table shows the requested "pay" for a request the gate refused.
- Lessons come from 42 training cases with simulated notes and the same case types as the test set; a stronger prompted baseline (e.g. multi-step) could narrow the gap.

7 Conclusion

A harness around Apertus 8B kept payments within each client's signed mandate on synthetic data: on the blind confirm set, wrong payments fell from 178 of 300 (Apertus alone) to 0 of 300, at the cost of holding 7 of 200 legitimate

invoices for a person, and all 561 attack attempts on the agent's identity and authority were blocked. Apertus does the reading and the review of content; code, the gate and people decide what money may move, and the signed ledger lets anyone check it afterwards. The result is evidence for the mechanism, not a production error rate.

Directions we would consider, not commitments (none started): mandates in a standard verifiable-credential format and keys in an HSM; reading structured invoice data (QR-bill, e-invoices) directly so the model reads less; supplier profiles or per-field confidence to hold fewer legitimate invoices; a sign-off step for promoting lessons to code; measuring reviewer accuracy and workload; testing on real, anonymised payment runs with a fiduciary.

8 Reproducibility

`make run` in `track_2b/` on a clean checkout builds the image, replays the recorded Apertus responses (no key needed), writes `results/`, prints the table and serves the app on port 8000; `make eval` prints and exits; `python -m agent holdout` replays the holdout. `LLM_CACHE_PATH=data/cache/study_responses.jsonl python -m agent study tune` replays the study (`STUDY_FORCE=1 … study final|confirm`, `study baseline-gen1`); the final set's blind results are kept in `results/study/final_blind.json`. GitHub Actions runs the tests, builds the image and re-runs `eval` and `holdout` without a key on every push, and fails if the metrics, risk model or holdout differ from the committed files. Data generation is deterministic; no GPU needed for replay. Commit: `6b8625b` (code and results; later commits change only the report and the demo video link).

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. Code: Apache-2.0 (repository `LICENSE`); data: CDLA-Permissive-2.0. Third-party components: Apertus v1.5 8B (Apache-2.0), FastAPI (MIT), Uvicorn (BSD-3-Clause), cryptography (Apache-2.0 OR BSD-3-Clause).

References

- Apertus model family: https://huggingface.co/swiss-ai
- KYA-OS, Decentralized Identity Foundation (donated by Vouched): https://blog.identity.foundation/kya-os/; reference implementation `@kya-os/mcp` (MIT): https://github.com/decentralized-identity/kya-os-mcp
- ISO 20022 pain.001.001.09 (customer credit transfer initiation)
- FINMA Guidance 08/2024 on governance and risk management when using AI
- Valindex (2026-02), count of Swiss firms in NOGA 6920 — secondary source